NaijaBiz AI: Technical Documentation

Oct 12, 2026 · @Usama

Overview and status

NaijaBiz AI turns a merchant's voice, receipt photo or text into a proposed business action (add stock, record a sale or expense, answer a question). The merchant confirms it, and only then does the backend save it. This document describes the system and documents the AI service (the "ML service") in detail, because that is the part built so far. The backend and frontend sections describe the agreed contract, not finished code.

Component	Owner	Status
ML service (FastAPI): speech, command parsing, receipts, Ask My Business, alerts, forecasts	ML engineer	Built and tested with mock models. Real N-ATLaS models load and serve from the Colab notebook; accuracy per language not yet measured
N-ATLaS language and speech models	Awarri / Federal Ministry (open models)	Loaded and used from Colab for development; production hosting not yet set up
Backend API and database	Backend engineer	Contract drafted (see the project description); implementation status to be filled in by the backend engineer
Frontend app	Frontend engineer	Screens and payloads agreed; implementation status to be filled in by the frontend engineer
Product image recognition	ML engineer	Not built. Prototype capability only

Languages: English (Nigerian-accented), Yoruba, Hausa and Igbo.

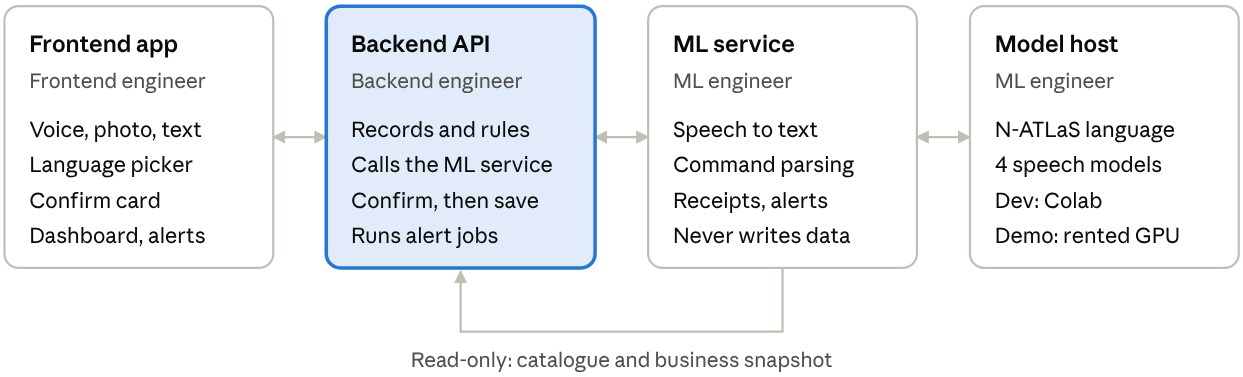
Design rules that hold everywhere

1. The AI proposes; the merchant confirms; the backend saves. The ML service never writes to the database.
2. Money amounts are parsed by plain rules, not by the language model.
3. Every AI answer about the business is built from real figures supplied by the backend, never guessed.
4. If the language model is unavailable, the service still works through rule-based fallbacks in English.

System architecture

The frontend talks only to the backend. The backend calls the ML service, which calls the models and returns a proposal. The ML service never saves business data.

The ML service proposes and only the backend saves

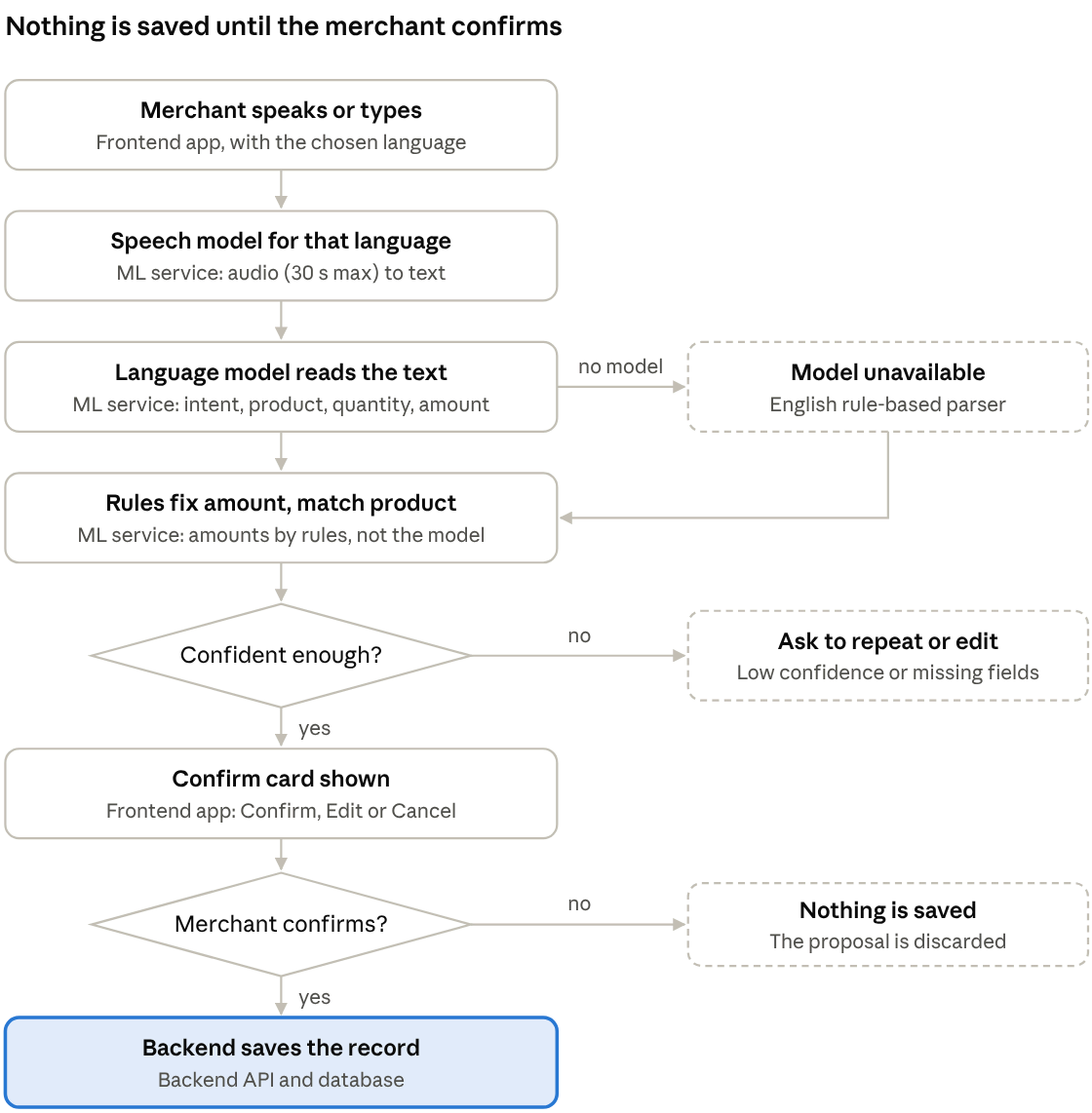


system architecture · 4 components

The ML service reads the merchant's catalogue and business figures from the backend (read-only). It reaches the model host over HTTP, so the models can move from Colab to a rented GPU by changing settings, not code.

Request flow

A spoken command passes through six steps and two checks. A typed command skips the speech step. A receipt photo takes the same route, with OCR in place of speech and a review screen in place of the confirm card.



voice command to saved record · 6 steps, 2 decisions

The low-confidence check uses `MIN_CONFIDENCE` (0.6) and missing required fields. If the language model cannot be reached, the English rule-based parser still returns a proposal, but it is always low confidence for actions, so the merchant is asked to check it.

ML service modules

The service is a Python FastAPI app in `ml_service/`. Routers are thin; the logic lives in `app/services/`.

File	Responsibility
`app/main.py`	Creates the app, mounts the routers, checks the `X-API-Key` header, exposes `/health`
`app/config.py`	Reads every setting from environment variables
`app/schemas.py`	The shared data contract: `ActionProposal`, `ReceiptResponse`, `BusinessSnapshot`, and the request and response models
`app/services/asr.py`	Speech to text. Modes: mock, local models, or a remote server. One N-ATLaS speech model per language
`app/services/llm.py`	Client for any OpenAI-style chat server (the notebook, vLLM or a hosted API). Retries when the model returns invalid JSON
`app/services/intent_parser.py`	Turns text into intent, entities and confidence. The model extracts, then rules correct the numbers and match the product
`app/services/product_matcher.py`	Fuzzy-matches what was said ("coke") to the merchant's catalogue ("Coca-Cola")
`app/services/receipt_reader.py`	OCR plus language model to read an invoice, then validates totals and flags problems
`app/services/ask_business.py`	Ask My Business: selects the relevant real figures, then has the model phrase the answer
`app/services/insights.py`	Alert rules: low stock, sales drop, expense growth, overdue debt
`app/services/forecasting.py`	Days of stock left, restock advice, cash-flow average, affordability, health score
`app/services/language.py`	Confirmation messages in the merchant's language
`app/services/backend_client.py`	Read-only calls to the backend (catalogue and business snapshot)
`app/utils/naira.py`	Parses "4 million naira", "120k", "₦80,000" and spoken numbers

File	Responsibility
`app/utils/audio.py`	Converts uploaded audio to 16 kHz mono WAV with ffmpeg
`app/prompts/*.md`	The prompts for command parsing, receipts and Ask My Business, kept as editable files

API reference

Base URL is wherever the service runs (default `http://localhost:8080`). When `ML_API_KEY` is set, every request, including `/health`, must send it in the `X-API-Key` header. Interactive documentation is served at `/docs`.

Endpoint	Request	Response
`GET /health`	none	Status and which backends are active (mock, transformers, remote)
`POST /transcribe`	Form: audio file, `language` (en, yo, ha, ig)	`{transcript, language, confidence}`
`POST /interpret`	JSON: `{text, language, business_id?, catalogue?}`	`ActionProposal`
`POST /interpret/audio`	Form: audio file, `language`, `business_id?`, `catalogue_json?`	`ActionProposal`, from speech in one call
`POST /parse-receipt`	Form: image file, `business_id?`, `catalogue_json?`	`ReceiptResponse`
`POST /ask`	JSON: `{question, language, snapshot or business_id}`	`{answer, language, facts_used}`
`POST /insights`	JSON: `{snapshot}`	`{alerts: [...]}`
`POST /forecast`	JSON: `{snapshot, purchase_price?}`	Stock forecasts, cash flow, health score, optional affordability

ActionProposal

This is what the AI understood. It is a proposal only.

Field	Meaning
`intent`	`add_inventory`, `record_sale`, `record_expense`, `record_debt_payment`, `query` or `unknown`
`language`	The language the merchant used
`transcript`	The text that was understood (typed or transcribed)
`entities`	`product`, `matched_product_id`, `matched_product_name`, `quantity`, `unit`, `total_amount` (naira), `unit_cost` (naira per unit), `customer`, `category`, `paid`
`confidence`	0 to 1
`low_confidence`	True when the app should ask the merchant to repeat or edit before confirming
`missing_fields`	Required fields the AI could not find
`confirmation_message`	Text to show on the confirm card

Example request and response:

```
POST /interpret
{"text": "I just bought 50 bags of rice for 4 million naira", "language":
"en",
 "catalogue": [{"id": "p1", "name": "Rice"}]}

{"intent": "add_inventory", "language": "en",
 "transcript": "I just bought 50 bags of rice for 4 million naira",
 "entities": {"product": "rice", "matched_product_id": "p1",
"matched_product_name": "Rice",
              "quantity": 50, "unit": "bag", "total_amount": 4000000,
"unit_cost": 80000},
 "confidence": 0.55, "low_confidence": true, "missing_fields": [],
 "confirmation_message": "Add 50 bags of Rice at ₦80,000 each?"}
```

The example above is from the rule-based fallback (no language model connected), which reports a confidence of 0.55 and so is marked low confidence. With the language model connected the confidence comes from the model.

Receipt response

`supplier`, `items` (each with `name`, `quantity`, `unit`, `unit_price`, `line_total`, `matched_product_id`), `total`, `currency` (NGN), `confidence`, `warnings`, `raw_text` and `confirmation_message`. Warnings are raised when no items are found, when the item lines do not add up to the total, or when quantity times price does not match a line total. The app should show these on the review screen.

Errors

Code	When
400	Empty file, audio longer than 30 seconds, unreadable audio, or the speech server is unreachable or rejected the request (the message says which)
401	Missing or wrong `X-API-Key`
422	Invalid request body, or `/ask` called with neither `snapshot` nor `business_id`
502	`/ask` could not load business data from the backend

Models, hosting and configuration

N-ATLaS models

The language and speech models come from the open N-ATLaS project by Awarri Technologies and Nigeria's Federal Ministry of Communications, Innovation and Digital Economy. All five repositories are gated: accept the terms on each Hugging Face page and use a read token (`HF_TOKEN`).

Repository	Purpose	Notes
`NCAIR1/N-ATLaS`	Language model: command understanding, question answering, translation	Llama-3 8B fine-tune. Text only. English, Hausa, Igbo and Yoruba. About 8,092 tokens of context. About 16 GB of GPU memory in 16-bit, or 5 to 6 GB in 4-bit
`NCAIR1/NigerianAccentedEnglish`	Speech to text, English	Whisper Small, about 244M parameters
`NCAIR1/Yoruba-ASR`	Speech to text, Yoruba	Whisper Small, about 244M parameters
`NCAIR1/Hausa-ASR`	Speech to text, Hausa	Whisper Small
`NCAIR1/Igbo-ASR`	Speech to text, Igbo	Whisper Small

Speech models take 16 kHz audio and at most 30 seconds per clip. The merchant's language picker selects which speech model runs, so no language detection is needed. The Yoruba model page states the Whisper Small base, the 16 kHz input and the 30-second limit; the other three speech models are described as Whisper-based in the project announcement, so confirm their size and limits on each model page.

Licence: the N-ATLaS terms allow free use for projects with up to 1,000 active users in 30 days and require attribution to Awarri Technologies and the Federal Ministry. Larger deployments need a commercial licence.

Speech modes (`ASR_BACKEND`)

Value	What happens
`mock`	Returns a fixed demo phrase. Default, so the rest of the pipeline can be built without models
`transformers`	Loads the language's model inside the ML service on first use and keeps it in memory

Value	What happens
`remote`	Sends the audio to an OpenAI-style `/v1/audio/transcriptions` server, using `ASR_BASE_URL` and `ASR_API_KEY`, or the `LLM_*` values if those are empty

Development hosting: Colab notebook

The notebook `NATLaS_colab.ipynb` loads all five models and, in section 5, serves them through a Cloudflare tunnel:

Endpoint	Purpose
`POST /v1/chat/completions`	Text generation with N-ATLaS
`POST /v1/audio/transcriptions`	Speech to text; the `language` field (en, yo, ha, ig) picks the model
`GET /health`	Lists the loaded speech languages

Both model endpoints require `Authorization: Bearer <SECRET>`, and a lock allows one GPU request at a time. Colab is for development only: sessions time out, the GPU can be reclaimed, and the tunnel address changes on every start. For the demo, host the models on a rented GPU (language model with vLLM, speech models in the ML service) behind a stable address.

Settings

All settings are environment variables; a template is in `.env.example`.

Variable	Default	Meaning
`ML_API_KEY`	empty	Shared secret the backend sends in `X-API-Key`. Empty disables the check (development only)
`BACKEND_BASE_URL`, `BACKEND_API_KEY`	`http://localhost:8000`, empty	Where the service reads catalogue and snapshot data

Variable	Default	Meaning
`LLM_BACKEND`	`mock`	`mock` (rules only) or `openai_compatible`
`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`	`http://localhost:8001/v1`, `EMPTY`, `NCAIR1/N-ATLaS`	The chat server
`ASR_BACKEND`	`mock`	`mock`, `transformers` or `remote`
`ASR_MODEL_EN`, `_YO`, `_HA`, `_IG`	the four NCAIR1 speech repos	Model per language
`ASR_BASE_URL`, `ASR_API_KEY`	empty	Remote speech server; fall back to the `LLM_*` values
`HF_TOKEN`	empty	Hugging Face read token
`OCR_BACKEND`	`mock`	`mock` or `tesseract`
`MIN_CONFIDENCE`	`0.6`	Below this, a proposal is marked low confidence

Business logic

All of this is plain, tested code in `forecasting.py` and `insights.py`. The thresholds are starting heuristics to tune on the demo data; they are not validated against real businesses.

Command parsing

1. The language model returns intent, entities and a confidence.
2. Rules then override the amount: the money value found by `naira.py` replaces the model's `total_amount` for add-stock, sale, expense and debt-payment actions.
3. For add-stock, `unit_cost` is the total divided by the quantity.
4. The product is fuzzy-matched to the catalogue (match score of at least 0.6, with a small alias list such as coke to Coca-Cola).

5. If a required field is missing, confidence is capped at 0.5. Unknown intents are capped at 0.3. A proposal is `low_confidence` when confidence is under `MIN_CONFIDENCE` (0.6) or any field is missing.

Required fields: product and quantity for add-stock and sale; total amount for an expense; customer and amount for a debt payment.

When no language model is connected, a small English rule-based parser handles bought, sold and paid sentences. It reports a fixed confidence (0.55 for actions, 0.5 for questions, 0.2 for unknown), so its results are always marked low confidence for actions.

Stock forecast

Quantity	Rule
Days of stock left	Current stock divided by average daily sales. Not calculated if the product is not selling
Restock advice	Given when days left is under 5
Low-stock alert	Stock at or below the product's minimum, or under 5 days left. Severity is critical under 2 days, otherwise warning

Alerts (`/insights`)

Alert	Trigger
`low_stock`	See above
`sales_drop`	This week's sales are at least 15% below the usual weekly average
`expense_growth`	Expenses grew at least 10 percentage points faster than revenue compared with last month
`overdue_debt`	One or more customers have owed money for more than 30 days (summarised in one alert)

Cash flow and affordability

- **Average monthly cash flow:** the mean of the last three monthly net cash-flow values. The next-month estimate is currently the same number.
- **Affordability:** safe spending is up to half of the average monthly net cash flow. The safe range runs from 80% to 100% of that, rounded to the nearest ₦1,000. A purchase fits if its price is within the upper figure. If average cash flow is zero or negative, the answer is no.

Health score

A 0 to 100 score starting at 50: up to 25 points added or removed for profit margin this month, up to 15 removed for customer debt relative to monthly revenue, up to 15 removed for the share of selling products that are low on stock, and 5 added if revenue this month is at least last month's. The weights are placeholders.

Ask My Business

A keyword router picks which real figures to include (stock, debts, finance, or an overview). The language model only phrases an answer from those figures and is told not to invent numbers. Without a language model, an English template answer is returned. The router handles simple questions only; history questions such as best sales day or top five products need extra data from the backend.

Contract with backend and frontend

The ML service is stateless. It depends on the backend for data and on the frontend for the merchant's language and confirmation.

Confirm-then-commit

1. The frontend sends voice, a photo or text to the backend, with the merchant's language.
2. The backend calls the ML service and receives an `ActionProposal` (or a receipt result).
3. The frontend shows `confirmation_message` with Confirm, Edit and Cancel. If `low_confidence` is true, it asks the merchant to check the details or repeat.
4. Only on Confirm does the backend write to the database. The backend should keep a log of every proposal and its outcome.

What the backend provides to the ML service

These two read-only endpoints are proposals for the backend engineer to confirm. They are called from `backend_client.py`.

Endpoint	Returns
`GET /products?business_id=`	A list of products with `id` and `name`, used to match what the merchant said
`GET /ai/snapshot?business_id=`	One `BusinessSnapshot` object, below

Alternatively the backend can send the catalogue or the snapshot inside the request, which avoids the extra call.

BusinessSnapshot fields

Field	Meaning
`products`	Per product: `id`, `name`, `unit`, `stock`, `min_stock`, `avg_daily_sales`, `selling_price`, `cost_price`
`sales_this_week`, `avg_weekly_sales`	This week's sales and the average of previous weeks (naira)
`revenue_this_month`, `revenue_last_month`	Naira
`expenses_this_month`, `expenses_last_month`	Naira
`monthly_net_cashflow`	Net cash flow per month, oldest first
`debts`	Per debt: `customer`, `amount`, `days_outstanding`

The backend owns the calculation of these figures, in particular `avg_daily_sales`, because the forecasts depend on them.

What the frontend needs

- A language picker (en, yo, ha, ig) saved on the merchant profile, sent with every voice or text request.

- Audio clips of 30 seconds or less.
- A confirm card driven by `confirmation_message`, `entities`, `low_confidence` and `missing_fields`, with editable fields.
- A review screen for receipts that shows `warnings`.

The full draft data model and endpoint list for the backend is in the project description document.

Testing and evaluation

Automated tests. `pytest` in `ml_service/` runs 36 tests with all models in mock mode. They cover amount parsing, command parsing and product matching, forecasting, alert rules, every API endpoint, and the remote speech client (including a 30-second-limit error and an unreachable server). They check the logic, not the quality of the real models.

Model quality is measured separately, in the notebook and scripts.

Tool	What it measures
Notebook section 4b	For each test phrase, two strategies (parse the local-language text directly, or translate to English first): share of valid JSON, correct intent, product, quantity and amount, per language
Notebook section 4c	Word error rate of each speech model against what the speaker said, plus the parsed result of each transcript
`scripts/eval_intent.py`	Intent and entity accuracy per language, from `data/eval/phrases.csv`
`scripts/eval_asr.py`	Word error rate per language, from `data/eval/asr.csv` and recordings
`scripts/eval_receipt.py`	Item and total accuracy on receipt photos with a matching `.json` of expected values

Results so far. None measured on the real models yet. The only phrases in the test set are six English examples, and the 100% accuracy they show comes from the rule-based fallback on those six rows. Before the demo, add 10 to 15 native-speaker phrases and recordings per language, run both strategies, and record the figures here.

Language	Speech word error rate	Intent accuracy (direct)	Intent accuracy (translate first)	Notes
English				
Yoruba				Published evaluations show weaker Yoruba text quality
Hausa				
Igbo				

The notebook counts unparseable model output as wrong for every field a phrase expects, so a model that often breaks the JSON cannot look better than it is.

Security, limitations and roadmap

Security and privacy

- **Access:** when `ML_API_KEY` is set, all endpoints require it in `X-API-Key`. It must be set on any deployed copy; leaving it empty is for local development only. The notebook server uses a separate bearer secret.
- **No writes:** the ML service holds no database credentials and never writes business data. Its only calls to the backend are reads.
- **Audio and images:** the ML service does not store uploaded audio or receipt images. The notebook server writes each audio clip to a temporary file and deletes it after reading.
- **Confirmation:** nothing the AI proposes changes data until the merchant confirms, which limits the damage from a misheard command.
- **Secrets:** keep `HF_TOKEN`, `ML_API_KEY` and the notebook secret out of source control. `.env.example` holds placeholders only.
- **Third parties:** if a hosted language model is used as a backup, the text of merchant commands and business figures is sent to that provider. Decide and disclose this before using it with real merchant data.
- **Not yet covered:** per-merchant authentication in the ML service (the backend is expected to authenticate users), rate limiting, and audit logging inside the ML service.

Known limitations

Limitation	Effect	Planned handling
Colab is not a stable host	Sessions end and the tunnel address changes	Move to a rented GPU before the demo
Yoruba text quality is weaker in published evaluations	More wrong or low-confidence proposals	Test Yoruba first; fall back to English or ask the merchant to confirm
No speech confidence score	The app cannot tell a poor transcript from a good one	Always show the transcript on the confirm card
Confirmation messages exist only in English	Yoruba, Hausa and Igbo merchants see English confirmations	Add native-speaker-reviewed templates in `language.py`
Speech clips limited to 30 seconds	Long commands are rejected	The frontend limits recording length
Receipt OCR (Tesseract) untested on real phone photos	Misread items or totals	Test on the demo invoice; add image clean-up or switch OCR; the review screen shows warnings
Ask My Business uses a keyword router	Only simple questions are answered well	Move to model tool-calling and add history data from the backend
Forecast and health-score thresholds are heuristics	Advice may not fit a given business	Tune on the demo dataset
Context limit of about 8,092 tokens	Very long prompts are not possible	Prompts are short by design
Licence limit of 1,000 active users in 30 days	Larger deployments need a commercial licence	Out of scope for the prototype

Roadmap

Not built, listed only for planning: product image recognition, spoken replies (the N-ATLaS speech models only recognise speech), automatic language detection, WhatsApp input, payments, and credit scoring.

Setup and run guide

Run the ML service with mock models (no GPU)

```
cd ml_service
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload --port 8080
pytest
```

Open `http://localhost:8080/docs` to try each endpoint. With the defaults, speech returns a demo phrase, receipts return a demo invoice, and commands are parsed by rules. This lets the backend and frontend integrate before the real models are connected.

Connect the real models through the Colab notebook

1. In Colab, choose a T4 GPU runtime. Accept the terms on all five NCAIR1 repositories on Hugging Face and add a read token as the Colab secret `HF_TOKEN`.
2. Upload `NATLaS_colab.ipynb` and run it from the top. The first cell checks access to each repository.
3. In section 5, set `SECRET`, run the server cell and the tunnel cell, and copy the printed address.
4. In `ml_service/.env` set:

```
LLM_BACKEND=openai_compatible
LLM_BASE_URL=<printed address>/v1
LLM_API_KEY=<your SECRET>
ASR_BACKEND=remote
```

5. Restart the service and test with a Yoruba, Hausa or Igbo phrase through `/interpret`, then a recording through `/interpret/audio`.

Run with models inside the service (GPU machine)

Install `requirements-ml.txt`, set `HF_TOKEN` and `ASR_BACKEND=transformers`, and serve the language model separately (for example with vLLM) behind `LLM_BASE_URL`. The `Dockerfile` installs ffmpeg and Tesseract, which audio conversion and receipt OCR need.

Before the demo

1. Move the models to a rented GPU and use its stable address.
2. Set `ML_API_KEY` and give the key and address to the backend engineer.
3. Run the full demo script from a cold start and time the first request, since the first model load takes minutes.
4. Keep `LLM_BACKEND=mock` as the fallback if the model server fails, and record a backup video of the demo.